

## CHAPTER 5

# Terraform Variables & Outputs

*Making your infrastructure code dynamic, reusable, and safe to share*



Input variables • Output values • Sensitive data

# What We'll Cover

Four building blocks for flexible, secure Terraform configurations



## 5.1 Variable Types

String, number, boolean, list, map, object, tuple



## 5.2 Input Variables

Defaults, overrides, and .tfvars files



## 5.3 Output Variables

Returning resource details after apply



## 5.4 Sensitive Variables

# 5.1 Types of Variables in Terraform

Terraform supports seven core types for modeling any kind of input



## String

A single text value



## Number

Numeric values (int/float)



## Boolean

true or false



## List

An ordered array of values



## Map

Key-value pairs



## Object

Structured, named data



## Tuple

A fixed set of mixed types

Choose the type that matches your data shape — Terraform validates it on plan.

# Defining Variables

Declare each variable in **variables.tf**

Give it a description, a type, and an optional default value

- String default for instance\_type
- List of numbers for open ports
- Map of strings for resource tags

```
variables.tf

variable "instance_type" {
  description = "Type of EC2 instance"
  type        = string
  default     = "t2.micro"
}

variable "port_numbers" {
  type        = list(number)
  default     = [80, 443, 22]
}

variable "instance_tags" {
  type = map(string)
  default = {
    Name = "WebServer"
    Env  = "staging"
  }
}
```

# Using Variables in main.tf

Reference any declared variable with the var. prefix

```
main.tf

resource "aws_instance" "example" {
  ami           = "ami-0c55b159cbfafa1f0"
  instance_type = var.instance_type
  tags          = var.instance_tags
}
```



**Override a value at apply time**

```
$ terraform apply -var="instance_type=t3.medium"
```

## 5.2 Input Variables — Defaults & Overrides



### Has a Default Value

Optional at apply time — Terraform uses the default unless told otherwise.

```
variable "instance_type" { type    =  
string  default = "t2.micro"}
```



### Override with .tfvars

Create terraform.tfvars to set values without touching code:

```
instance_type = "t3.medium"
```

```
$ terraform apply -var-  
file="terraform.tfvars"
```

*No default set? Terraform requires an explicit value before it will apply.*

## 5.3 Using Output Variables

Outputs return important resource details after a deployment finishes

```
● ● ● outputs.tf

output "instance_public_ip" {
  description
    = "Public IP of instance"
  value =
    aws_instance.example.public_ip
}
```



### Expose Resource Data

Share IDs, IPs, or ARNs across modules



### Query Anytime

Run terraform output to view results



### Chain Configurations

Feed one module's output into another



### Fetch outputs after deployment

\$ terraform output



## 5.4 Sensitive Variables

*By default Terraform prints variable values in logs — mark secrets sensitive to hide them from CLI output.*

```
variables.tf
```

```
variable "db_password" {  
    type      = string  
    sensitive = true  
}
```

```
outputs.tf
```

```
output "database_password" {  
    value      = var.db_password  
    sensitive = true  
}
```



Terraform hides sensitive values in CLI output — plans, applies, and terraform output all show (sensitive value) instead of the real secret.



# Walkthrough — Dynamic VPC CIDR Block

Putting variables and outputs together to build a VPC with a configurable CIDR range

## 1 Define Variables — variables.tf

```
variable "vpc_cidr" {  
  type = string  
}  
  
variable "environment" {  
  default = "staging"}
```

## 2 Create the VPC — main.tf

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_vpc" "my_vpc" {  
  cidr_block = var.vpc_cidr  
  
  tags = {  
    Name = "MyVPC-${var.environment}"  
    Environment = var.environment  
  }  
}
```



# Walkthrough — Apply & Retrieve

## 3 Define the Output — outputs.tf

```
output "vpc_id" {  
  value = aws_vpc.my_vpc.id  
}
```

## 4 Apply with a CIDR Value

```
$ terraform init  
$ terraform apply -var="vpc_cidr=10.0.0.0/16"  
\  
-auto-approve
```

## 5 Retrieve the VPC ID

```
$ terraform output vpc_id  
vpc-0abc123def4567890
```



*Result: the CIDR block is dynamically assigned — no hard-coded values anywhere.*

# Chapter 5 Recap

*What to remember about Terraform variables and outputs*



## Seven variable types

string, number, bool, list, map, object, tuple



## Defaults & overrides

terraform.tfvars and -var keep config flexible



## Outputs surface data

IDs, IPs, and ARNs are available right after apply



## Sensitive = true

Keeps secrets like passwords out of CLI logs